Settl.ai

Pay-as-you-go for AI agents

Agents pay startups' APIs per use, or only for results that pass agreed checks, from a prepaid, capped budget settled on Stellar.

Velo x Noir Agentic Payment Hackathon · Design pack v1.0 · 3 November 2026
Solo build · Responsive web app · Stellar testnet

PART 1 Product Design

PART 2 UI Design and User Flows

PART 3 Implementation Plan

PART 1

Product Design

Pay-as-you-go for AI agents. Agents pay startups' APIs per use, or only for results that pass agreed checks, from a prepaid, capped budget settled on Stellar.

Naming: the brand is **Settl.ai**, shortened to "Settl" in body text. Packages are `settl-sdk` and `settl-client`, and the client function is `settlFetch()`. Outcome mode is marketed as **"Settl on Result"**.

- **Event:** Velo x Noir Agentic Payment Hackathon (build Nov 5, submit by Nov 8, 8:00 AM Manila)
- **Team:** solo
- **Status:** FINAL v1.0. Concept and design only; no code yet.
- **Platform:** responsive web app (desktop-first, works on a phone). No native mobile app.
- **Date:** 2026-11-03

---

1. One-liner and pitch

One-liner: Settl.ai is pay-as-you-go for AI agents: when an agent uses a startup's API, it pays per use from a prepaid, capped budget on Stellar, and can pay only for results that pass agreed checks. Startups earn from AI agents with five lines of code, with no subscriptions or invoices.

Pitch line: *"Agents pay as they go. Startups Settl."*

Registration form answers (submitted wording):

- *Idea:* "Settl.ai is pay-as-you-go for AI agents: when an agent uses a startup's API, it pays per use from a prepaid, capped budget on Stellar, and can pay only for results that pass agreed checks. Startups earn from AI agents with five lines of code, with no subscriptions or invoices, and companies never face a runaway bill."
- *Noir and Velo:* see §8. Each agent's identity is registered in `device_registry` and its limits in `agent_registry`. Its budget is loaded through Velo Checkout and confirmed by webhook into `payment_escrow`. Each use is paid with `authorize`, submitted only on a passing result in Settl on Result mode, with fees sponsored by Velo Gas Station. Providers collect with `claim`, and revoking an agent returns its unused budget.

30-second pitch: AI agents are becoming software's newest customers. They need to buy OCR, translation, data and models while they work. But SaaS billing assumes a human with a credit card, a signup form and a monthly plan, and Stripe doesn't support PH-registered businesses. Settl.ai makes any API pay-as-you-go for agents. An agent hits the API and gets back "402, pay ₱2". It checks its budget, pays from a prepaid escrow and gets the result. In outcome mode the startup gets paid only when the result passes checks both sides agreed on up front. Startups monetize agents from day one. Companies let agents spend without fearing a runaway bill.

---

2. The problem

For startups (providers), the hero side

- Usage-based billing is hard to build: metering, invoicing, failed cards, dunning.
- Agents can't complete human checkout flows (signup, card, plan, API key).
- PH startups have limited access to global usage-billing tools. Stripe doesn't support PH-registered businesses (to re-verify before pitching), and local gateways are built for cards and e-wallets, not machine payments.
- "Pay per outcome" pricing (per resolved ticket, per correct extraction) is in demand but needs a neutral way to decide what counts as a good outcome.

For companies running agents (buyers)

- Giving an agent a corporate card is unsafe: there are no caps per agent, no per-provider rules and no audit trail.
- Every new tool means a human signup and API-key handling, multiplied across tools and agents.
- They pay for garbage outputs because billing is per call, not per result.

---

3. The solution

Roles

Role	Example	What Settl gives them
Buyer (owns the agent)	Juan's Accounting Firm runs an AI bookkeeping agent	Agents that buy tools on their own, with on-chain caps, allowlists and an audit trail
Agent (the buyer's AI worker)	The bookkeeping agent processing 500 receipts	`settlFetch()`, which pays automatically, within policy
Provider (sells the API)	ResiboAI sells receipt OCR	Pay-per-use or pay-per-result revenue with no billing system
Verifier (Settl's judge)	Checks the OCR output	A neutral pass/fail decision before money moves

The AI agents are the **customers**. The provider's API may or may not be AI.

Analogy

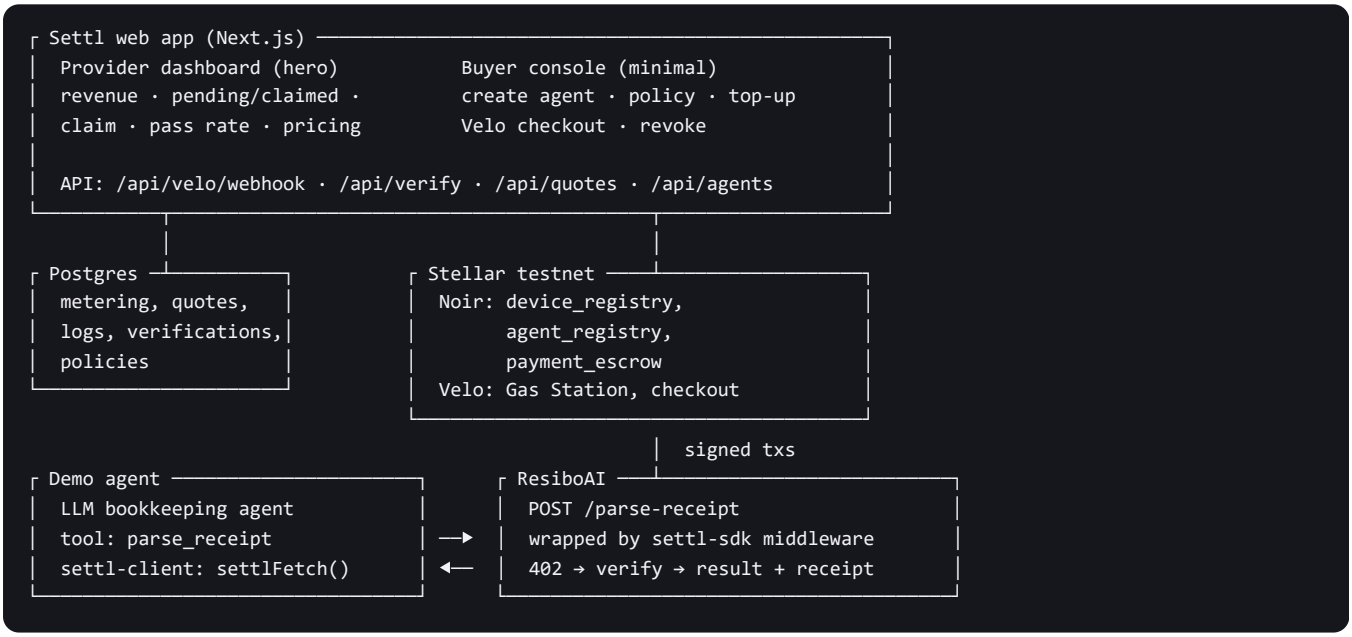
A highway toll. Startups own the roads (APIs). Agents are the cars. Escrow is the prepaid RFID load (like Autosweep or Easytrip). The policy is the fleet manager's rules. The verifier is a toll that only charges if the road got you where you needed to go.

---

4. How it meets the hackathon requirements

Requirement	How Settl meets it
Agentic payment: the agent decides, requests or makes the payment	The agent decides whether a tool is worth paying for and within budget. The verifier decides whether a result earned its payment. The provider's API *requests* payment through 402.
Built with Velo (`@carts1024/velo-sdk`)	Checkout sessions for buyer top-ups, signed webhooks to confirm funding, Gas Station to sponsor every Soroban transaction
Uses the latest Noir testnet contracts	`device_registry` gives each agent an identity, `agent_registry` enforces each agent's policy on-chain, `payment_escrow` holds prepaid funds, authorizations and claims
Registered on velo-build.dev	Pre-hackathon checklist (§17)

5. System overview



Components

Component	Type	Responsibility
`settl-sdk`	npm package (server)	Middleware: issues 402 quotes, verifies payment, attaches receipts, forwards outcome-mode results to the verifier
`settl-client`	npm package (agent side)	`settlFetch()` : parses the 402, checks local policy, signs or submits payment, retries
Settl web app	Next.js App Router	Provider dashboard, buyer console, webhook receiver, verifier service, settlement worker
Verifier	Service inside the web app	Runs deterministic checks for outcome mode and submits pre-signed authorizations on pass

Component	Type	Responsibility
ResiboAI	Demo provider API	Fake receipt OCR with canned, predictable outputs
Demo agent	Node script with an LLM	Bookkeeping agent whose `parse_receipt` tool uses `settlFetch`, with a visible reasoning log

Stack: TypeScript, Next.js (App Router), `@carts1024/velo-sdk`, `@stellar/stellar-sdk` (Soroban RPC), Neon Postgres, and AI SDK tool-calling for the demo agent.

---

6. Payment modes

6.1 Per-call mode (pay, then serve)

The agent pays before the provider does the work. Best for cheap, deterministic calls.

1. The agent sends `POST /parse-receipt` with no payment.
2. The SDK requests a quote from the Settl API, which stores it (nonce, price, expiry), and returns **402** with that quote (§7.1). Providers stay stateless.
3. `settlFetch` checks local policy: provider allowlist, `maxPrice`, daily budget.
4. The agent builds `payment_escrow.authorize(agent, device_hash, merchant, asset, amount, nonce)`, signs it and submits through **Velo Gas Station** (`gas.sponsorAndSubmit`, then `waitForResult`).
5. The agent retries with `X-PAYMENT` set to the payment proof (§7.2).
6. The SDK verifies the proof on-chain: the transaction succeeded, the arguments match the quote, and the nonce hasn't been used. It marks the quote paid and runs the handler.
7. The response includes the result and an `X-PAYMENT-RECEIPT` header.

Speed option (stretch): "optimistic" verification serves the response once the transaction *simulates* successfully and confirms in the background. Cheap calls only.

6.2 Outcome mode (serve, verify, then pay), the differentiator

The provider does the work, and money moves only if the result passes criteria agreed in the quote.

1. The agent sends a request with no payment, and the SDK returns **402** with `mode: "outcome"` and `criteria`.
2. The agent checks policy and **pre-signs** the Soroban authorization entry for `authorize(...)` without submitting it.
3. The agent retries with `X-PAYMENT` holding the pre-signed authorization.
4. The SDK runs the handler, then posts `{ quoteId, result, preSignedAuth }` to the Settl verifier.
5. The verifier runs the checks (§9):
   - **Pass:** it submits the pre-signed authorization through Gas Station. Money moves to the provider's pending balance in escrow, and the response is released with the receipt.
   - **Fail:** it discards the authorization and returns a **422**-style response with the reasons. The agent pays nothing, and the provider can contest the result (stretch).
6. Who gets the result if it fails: the MVP still returns it to the agent, marked "unverified, not charged". That keeps the demo honest and simple. Withholding it is a stretch option.

Dependency: outcome mode requires that someone other than the agent can submit the agent's signed authorization. See Spike S1 (§15). **Fallback:** the verifier returns `pass` to the agent client, which then submits

`authorize` itself. That's weaker, because the client could decline to pay, but fine for the demo.

---

7. Wire protocol (draft)

To align with Noir's or Velo's x402 conventions if either defines a canonical 402 body. Otherwise this is Settl's own format, modeled on x402.

7.1 `402 Payment Required` body

```
{
  "settl": "0.1",
  "quoteId": "q_01JC...",
  "mode": "outcome",
  "network": "stellar:testnet",
  "asset": "<escrow token contract id>",
  "amount": "2.0000000",
  "displayPrice": { "currency": "PHP", "amount": "2.00" },
  "merchant": "G...PROVIDER_WALLET",
  "escrowContract": "CA5S4S7QGHJHJWVJBYL3CXZXZTNGKXMNZVDAEQNN7NUXFP4D7BYK7HIX",
  "nonce": "184467",
  "expiresAt": "2026-11-06T10:05:00Z",
  "resource": "POST /parse-receipt",
  "description": "Parse one receipt image into structured JSON",
  "criteria": {
    "schema": "resibo.receipt.v1",
    "required": ["merchant_name", "date", "total", "line_items"],
    "minConfidence": 0.85,
    "lineItemsSumToTotal": { "tolerance": 0.01 }
  },
  "verifier": "https://settl.ai/api/verify"
}
```

7.2 `X-PAYMENT` header (base64 JSON)

```
// per-call
{ "quoteId": "q_...", "kind": "tx", "txHash": "abc123...", "agent": "G...AGENT", "deviceHash": "…" }
// outcome
{ "quoteId": "q_...", "kind": "auth", "authEntryXdr": "AAAA...", "agent": "G...AGENT", "deviceHash": "…" }
```

7.3 `X-PAYMENT-RECEIPT` header

```
{ "quoteId": "q_...", "status": "paid", "txHash": "...", "amount": "2.0000000",
  "verified": true, "remainingBudget": "48.0000000" }
```

7.4 Provider integration (developer experience target)

```
import { settl } from "settl-sdk";

app.post("/parse-receipt",
  settl({ price: "2.00", currency: "PHP", mode: "outcome", criteria: "resibo.receipt.v1" }),
  parseReceiptHandler
);
```

7.5 Agent integration

```
import { settlFetch } from "settl-client";

const res = await settlFetch("https://resibo.demo/parse-receipt", {
  method: "POST", body: image,
  wallet: agentKeypair, deviceHash, maxPrice: "5.00"
});
```

---

8. Noir and Velo mapping

8.1 Concept mapping

Noir models spenders as **devices**. Settl treats each **AI agent** as a device.

Noir concept	Settl meaning
`wallet` (owner)	Buyer organization's Stellar wallet
`device_hash`	`sha256("settl:agent:" + agentId)`
`agent` (address)	The AI agent's own Stellar keypair
`merchant`	Provider's payout wallet
`AgentPolicy { max_amount, asset, expires_at }`	On-chain hard limits for the agent
Escrow balance for each device	The agent's prepaid budget
Pending payments for each merchant	The provider's earned, unclaimed revenue

8.2 Testnet contract IDs ("latest Noir Testnet contracts" from the hackathon registration form, 2026-11-03)

- `device_registry`: `CB4DPMGOA374JIB2ZVD4AHW5GJKQUYNOFGRYNOJ75EFH2KCJSMHOWIRA`
- `agent_registry`: `CBTDMJVCFQDIVWZBKAKZ2FQ3UEJNAAUFYCTJ3E4ON2MYXPLPSIXQ25JZ`
- `payment_escrow`: `CA5S4S7QGHJHJWVJBYL3CXZXZTNGKXMNZVDAEQNN7NUXFP4D7BYK7HIX`

These replace the IDs in the Noir README (redeployed 2026-10-24), which are now outdated. They were transcribed from a screenshot, so copy them from the form at kickoff.

8.3 Call sequences

Create agent (buyer console)

1. Generate the agent keypair and compute `device_hash`.
2. `device_registry.register(org_wallet, device_hash, agent)`
3. `agent_registry.register_agent(org_wallet, device_hash, agent, max_amount, asset, expires_at)`

Top up the agent budget

1. `velo.checkout.sessions.create({ amount, asset, description, successUrl, cancelUrl })` returns `checkoutUrl`. The buyer pays there.

2. Velo webhook to `/api/velo/webhook`. Check it with `Velo.webhooks.verify(...)` and confirm through `paymentIntents.retrieve`.
3. The settlement worker calls `payment_escrow.fund_escrow(token, org_wallet, device_hash, amount)`.
4. **Hackathon simplification:** org wallets are custodial testnet keypairs managed by Settl, so the server can sign `fund_escrow` after the checkout confirms. A non-custodial version would use `@carts1024/velo-wallets` to let the buyer sign. See Open Question Q3.

Pay (per-call): `payment_escrow.authorize(...)`, signed by the agent and submitted through Gas Station. **Pay (outcome):** the same call, pre-signed by the agent and submitted by the verifier on pass. **Claim (provider dashboard):** `payment_escrow.claim(token, merchant)`, signed by the provider wallet. **Read state:** `balance_of(device_hash)`, `pending_balance(merchant)`, `agent_registry.get_policy(device_hash)`, `escrow_token()`.

Revoke an agent (safe order, per Noir docs)

1. `agent_registry.revoke_agent(...)`
2. `payment_escrow.sweep_on_revoke(token, device_hash, agent)`. This returns the leftover budget to the owner.
3. `device_registry.revoke(...)`, only after the escrow balance is 0. Revoking first breaks the refund lookup.

8.4 Velo usage summary

Velo feature	Used for
`checkout.sessions.create` / `paymentIntents.retrieve`	Buyer top-ups
`Velo.webhooks.verify`	Trusting top-up confirmations
`gas.sponsor` / `sponsorAndSubmit` / `waitForResult`	Fee-less agent payments, verifier submissions, claims
Idempotency keys and correlation IDs	Linking quote, payment and receipt; safe retries

---

9. Verifier (outcome mode)

9.1 Principles

- **Terms first:** the criteria travel inside the 402 quote, so both sides see them before any work.
- **Deterministic in the MVP:** no LLM judge on stage. Every check is reproducible.
- **Neutral:** the verifier is run by Settl, not by the buyer or the provider.
- **Explainable:** every fail lists the exact check that failed.

9.2 Check set for `resibo.receipt.v1`

Check	Pass condition
Schema	Output validates against the receipt JSON schema
Required fields	`merchant_name`, `date`, `total` and `line_items` are present and non-empty
Confidence	`confidence ≥ minConfidence` (0.85)
Arithmetic	The sum of `line_items[].amount` equals `total` within the tolerance

Check	Pass condition
Date sanity	`date` parses and isn't in the future

9.3 Results

- `pass`: submit the authorization and the provider is paid.
- `fail`: discard the authorization and store the reasons. It counts against the provider's pass rate.
- `error` (verifier fault): the agent isn't charged and the event is flagged for review.

9.4 Stretch

- An LLM judge for fuzzy criteria ("Is the summary faithful to the source?")
- A provider "contest" flow leading to a human review queue, filling the dispute gap in Noir
- Criteria templates for other services: translation, geocoding, lead enrichment

---

10. Policy model

Layer	Rule	Enforced by
On-chain	Spend cap (`max_amount` ; per payment or lifetime, see Spike S2)	`agent_registry.check_payment` inside `authorize`
On-chain	Allowed asset	`agent_registry`
On-chain	Expiry (`expires_at`)	`agent_registry`
On-chain	Budget can't exceed funded balance	`payment_escrow`
On-chain	Replay protection (nonce)	`payment_escrow`
Off-chain	Provider allowlist	`settl-client` and Settl API
Off-chain	Max price per call	`settl-client`
Off-chain	Daily budget	Settl API (metering)
Off-chain (stretch)	Human approval above a threshold	Settl API sends a notification and holds the request

Pitch point: even if the agent's code is compromised, the on-chain caps still hold.

---

11. Data model (Neon Postgres, draft)

Table	Key fields
`orgs`	id, name, role (`buyer` / `provider`), wallet_public, wallet_secret_enc (testnet custodial)
`agents`	id, org_id, agent_public, agent_secret_enc, device_hash, status, created_at
`policies`	agent_id, max_amount, asset, expires_at, daily_budget, max_price_per_call, allowlist (provider ids)

Table	Key fields
`providers`	id, org_id, merchant_wallet, display_name
`endpoints`	id, provider_id, route, price, currency, mode, criteria_id
`quotes`	id, endpoint_id, agent_id, nonce, amount, mode, criteria, status (`issued` / `paid` / `verified` / `failed` / `expired`), expires_at
`payments`	id, quote_id, kind (`tx` / `auth`), tx_hash, status, submitted_at, confirmed_at
`verifications`	id, quote_id, result (`pass` / `fail` / `error`), reasons (json), checked_at
`topups`	id, agent_id, velo_payment_intent_id, amount, status, fund_tx_hash
`claims`	id, provider_id, amount, tx_hash, claimed_at
`webhook_events`	id, source, event_id, payload, verified, processed_at (idempotency)

Nonce allocation: one sequence per (device, agent). The client sends payments one at a time per agent to avoid nonce races (see Q4).

---

12. Screens

Provider dashboard (hero)

- **Overview:** revenue today and all time, calls, pass rate, and a live feed of paid calls
- **Earnings:** pending (`pending_balance`) vs claimed, a **Claim** button and claim history with tx links
- **Endpoints:** price, mode (per-call / outcome), criteria, and an SDK snippet to copy
- **Quality:** verifier pass/fail breakdown by check (e.g. "12% failed the arithmetic check")
- **Customers:** revenue by buyer org and by agent

Buyer console (minimal)

- **Agents:** create, view balance, view policy, revoke (in the safe order)
- **Top up:** amount, then Velo checkout, then the balance updates after the webhook
- **Policy:** cap, expiry, allowlist, max price per call, daily budget
- **Activity:** each payment: provider, amount, verified or not, tx link

Demo agent console

- A terminal or web log showing the agent's reasoning: "*Need to parse receipt 17. ResiboAI quotes ₱2, outcome mode. Within budget (₱48 left) and on the allowlist. Paying.*"

---

13. Demo script (about 3 minutes)

1. **Setup (20s):** "Juan's firm has a bookkeeping agent with a ₱50 budget, capped on-chain. ResiboAI is a 2-person startup that added Settl in 5 lines."

2. **Happy path (40s):** the agent processes a clean receipt and gets a 402 quote for ₱2 in outcome mode. It pays, the verifier passes the result, and ResiboAI's dashboard ticks to +₱2 live, with a tx link.
3. **Bad result (30s):** the agent sends a blurry receipt. ResiboAI returns low confidence and totals that don't add up, so the verifier fails it. **₱0 charged**, and the reason is shown on both dashboards.
4. **Guardrail (30s):** the agent tries an endpoint that costs more than its cap. `check_payment` rejects it on-chain, and the policy holds even though the agent "wanted" to pay.
5. **Top-up (20s):** the buyer tops up through Velo checkout. The webhook arrives and the escrow balance updates.
6. **Claim (20s):** ResiboAI clicks Claim, one transaction runs and the money lands in its wallet.
7. **Close (20s):** "Any startup can sell to agents from day one, and only get paid for real results. Agents paying agents, settled on Stellar."

Backup: record the whole demo video on Nov 7 in case testnet or Wi-Fi fails on stage.

---

14. Trust model and known limitations

Limitation	Impact	MVP handling	Future fix
The owner can `defund_escrow` at any time	No hard guarantee for the provider in outcome mode	Show `balance_of` as visible solvency; track buyer reputation	A lock contract that holds committed funds for each quote
Noir doesn't restrict the merchant	An agent could pay any merchant	Off-chain allowlist in the client and API	A policy wrapper contract with an on-chain allowlist
Noir has no dispute function	Contested results have no on-chain path	The verifier is the decider; a contest queue is a stretch goal	An arbitration contract
Custodial org wallets	Not production-grade	Testnet only, clearly labeled	Non-custodial through Velo wallets
Roughly 5s ledger close	Per-call latency	Outcome or optimistic modes; batching is a stretch goal	Payment channels or batched settlement
Velo and Noir are alpha, testnet-only	No real money yet	Hackathon scope	Mainnet once Velo settlement ships, plus Velo's PH provider rails for PHP cash-out
Verifier centralization	Both sides trust Settl	Deterministic, explainable checks	Multiple verifiers, or on-chain attestations

---

15. Technical spikes (day 1, before building on assumptions)

#	Question	Why it matters	Fallback
S1	Can a third party submit a transaction carrying the agent's signed authorization for `authorize`?	Outcome mode depends on it	The agent client submits after a verifier pass
S2	Is `max_amount` a cap per payment or a lifetime total?	Console labels and guardrail demo	Label it accordingly and add an off-chain total

#	Question	Why it matters	Fallback
S3	Does Velo Gas Station sponsor Soroban contract invocations, not only classic operations?	Fee-less UX	Fund agents with a little testnet XLM through Friendbot
S4	Which token is `escrow_token()` on testnet (XLM SAC, test USDC, other)?	Pricing and top-ups	Price in that token and show PHP as display only
S5	Does `fund_escrow` require the device owner's signature, or can anyone fund?	Top-up flow design	Custodial org wallet signs (current plan)
S6	Nonce rules: strictly increasing, or just unique?	Concurrency and quote design	Send payments one at a time per agent
S7	Where does a Velo checkout payment settle (destination and asset)?	Bridging checkout to `fund_escrow`	A treasury wallet forwards to the org wallet

---

16. Scope

MVP (must ship)

- `settl-sdk` middleware with per-call and outcome modes
- `settl-client` (`settlFetch`) with allowlist and max-price checks
- ResiboAI demo API (canned clean and blurry outputs)
- Deterministic verifier for `resibo.receipt.v1`
- Provider dashboard: overview, earnings plus Claim, quality
- Buyer console: create agent, set policy, Velo top-up, revoke
- Demo agent with a visible reasoning log
- Full flow on Stellar testnet with the latest Noir contracts

Stretch (in order)

1. Optimistic verification for per-call mode
2. Human spend approval: a phone-sized web page opened from a notification ("Juan's Bookkeeper wants to pay ₱12 to GeoPH. Approve?"), used for the on-stage approval moment
3. Provider contest and review queue
4. LLM judge criteria
5. Lock contract for committed outcome-mode funds
6. Agent-to-agent demo: ResiboAI's own agent buys translation through Settl

Cut order if behind

Daily budget, then pricing editor, then buyer console polish, then the quality page. **Never cut:** 402 → pay → verify → claim.

---

17. Timeline

When	Goal
Nov 3–4	Register: Google form (closes Nov 4, 11:59 PM Manila), velo-build.dev project, star the Velo and Noir repos, follow the required X accounts, upload screenshots. Read the contract source for S1–S7. **No product code before kickoff** (check the rules).
Thu Nov 5	Kickoff. Run spikes S1–S7 on testnet. Scaffold the monorepo. End-to-end per-call payment: SDK, client and ResiboAI.
Fri Nov 6	Buyer console: agent creation, policy, Velo top-up through webhook to `fund_escrow`. Demo agent with the LLM and reasoning log.
Sat Nov 7	Outcome mode and verifier. Provider dashboard with Claim. Record the backup demo video.
Sun Nov 8, before 8:00 AM	Polish, README, architecture diagram, rehearse, submit. 11:00 AM top 10 announced, 1:00 PM pitch.

---

18. Business model (for the pitch)

- **Per-call mode:** 1% fee on settled volume.
- **Outcome mode:** a small fee per verification (the verifier is the value).
- **Later:** hosted criteria templates, verifier SLAs, and PH fiat cash-out through Velo's provider rails once live.

---

19. Decisions

#	Question	Decision
D 1	Product name	**Settl.ai** (short form "Settl"). Check the settl.ai domain and @settl_ai handle before launch.
D 2	Platform	**Responsive web app**, desktop-first. Providers and buyers are developers and finance or ops staff who work at a desk, and judges open a link with nothing to install.
D 3	Demo agent LLM	Set by the `AGENT_MODEL` env var; any AI SDK provider you have a key for.
D 4	Org wallets	Custodial testnet keypairs for the MVP (labelled as testnet only).
D 5	Concurrency	One payment at a time per agent.
D 6	Hosting	Vercel (web app and ResiboAI) with Neon Postgres.
D 7	Demo provider	Fake ResiboAI receipt-OCR API with canned outputs.
D 8	Team and hero side	Solo. Provider side leads the pitch.

#	Question	Decision
D9	Verifier	Deterministic checks in the MVP; LLM judge is a stretch goal.

PART 2

UI Design and User Flows

- **Parent spec:** `2026-11-03-settl-design.md`
- **Status:** FINAL v1.0 (user flows approved; the clickable mockup is a separate artifact)
- **Decisions:**
  - **Platform: responsive web app**, desktop-first and usable on a phone. No native mobile app.
  - Fintech-clean look: white, indigo `#5B5BD6`, Inter, soft rounded cards
  - Light default with a dark toggle
  - All four surfaces in scope: provider dashboard, buyer console, stage view, landing page
  - No existing design system; Settl gets its own style
  - Every screen designed to fit a 1280×720 projector
  - Stretch: a phone-sized spend-approval page for the on-stage approval moment (flow B5)

---

1. User flows

Roles

Role	Human or system	Surface
Provider (ResiboAI, a startup selling an API)	Human	Provider dashboard
Buyer (Juan's Accounting, a company running agents)	Human	Buyer console
Agent (Juan's bookkeeping AI)	System, no UI	Shown through the activity feed and stage view
Verifier (Settl's judge)	System, no UI	Shown as check results
Visitor / judge	Human	Landing page and stage view
Presenter (you, on stage)	Human	Stage view, plus switching surfaces

MVP has no login. A switcher in the top-left changes between Provider, Buyer and Stage. Each org's wallet is created and managed by Settl on testnet.

1.1 Provider (ResiboAI)

P1. Onboard and earn the first peso

```
Landing → "I sell an API" → Provider dashboard (empty state)
  → Create payout wallet (auto, testnet) ── "Settling on Stellar…" ──▶ ✓ Wallet ready
  → Add endpoint
      route: POST /parse-receipt
      price: ₱2.00
      mode: ( ) Per-call   (•) Pay-per-result
      criteria: [Receipt v1 ▾] → shows the 5 checks that will run
  → Copy SDK snippet (5 lines) + provider key
  → "Send a test request" (built-in sandbox agent)
  → Live feed shows the first call moving through the lane → ✓ Paid ₱2
```

Done when: the provider sees their first paid call without leaving the dashboard.

P2. Daily check-in

```
Overview → stat cards (Earned today · Pending claim · Calls · Pass rate)
  → Live feed row → click → Call detail drawer
      Lane: Quote ✓ → Policy ✓ → Pay ✓ → Verify ✓/✕ → Settled
      Quote: price, mode, nonce, expiry
      Payment: tx hash ↗ stellar.expert
      Verifier: checklist (✓ schema, ✓ fields, ✕ confidence 0.62 < 0.85 …)
      Customer: Juan's Accounting · agent: Juan's Bookkeeper
```

P3. Claim earnings

```
Earnings → Pending ₱86.00 → [Claim ₱86]
  → Confirm dialog: "Move ₱86 from escrow to your wallet G…X4F"
  → "Settling on Stellar… ~5s" (button disabled, progress)
  → ✓ "₱86 received" + tx ↗ → Pending becomes ₱0, the row appears in claim history
  ✕ on failure: "Claim didn't go through. Nothing moved." + [Try again] + error detail
```

P4. Improve quality

```
Quality → "Top failing checks: Confidence 7 · Totals 3 · Missing date 1"
  → click "Confidence" → list of failed calls → open one → see the input and output
  → (stretch) [Contest] → reason → review queue
```

P5. Change pricing or mode

```
Endpoints → endpoint card → Edit → price / mode / criteria → Save
  → Note: "Applies to new quotes only. Quotes already issued keep their price."
```

1.2 Buyer (Juan's Accounting)

B1. Onboard and launch an agent

```
Landing → "I run AI agents" → Buyer console (empty state)
  → Create org wallet (auto, testnet) ── ✓
  → New agent
      name: Juan's Bookkeeper
      Enforced on-chain:  cap ₱10 per payment · asset · expires Dec 31
      Enforced by Settl: allowed providers [ResiboAI] · max ₱5 per call · daily budget ₱20
  → Create → 2-step progress:
      ① Register agent identity (device_registry) ✓
      ② Set on-chain policy (agent_registry)     ✓
  → Credentials (shown once): agent secret + device hash
      [Copy] [Download .env]  ⚠ "Store this now. We won't show it again."
  → Prompt: "Your agent has ₱0. Top up to start." → B2
```

B2. Top up the budget

```
Agent detail → [Top up] → amount (₱50 · ₱100 · custom)
  → Redirect to Velo checkout (external) → pay
  → Return to Settl → "Waiting for payment confirmation…"
      (Velo webhook → fund_escrow on-chain)
  → ✓ "₱50 added to escrow" + tx ↗ → budget meter updates
  ✕ Cancelled at checkout → "Top-up cancelled. No charge."
  ⏳ Webhook slow (>30s) → "Still confirming. You can leave this page; we'll update the balance."
```

B3. Monitor spending

```
Agents list (cards: budget meter, status, spent today)
  → Agent detail → Activity
      ✓ Paid       ResiboAI  ₱2
      ○ Not charged ResiboAI ₱0  (verify failed: confidence)
      ⛔ Blocked    ResiboAI-Pro ₱15 (over on-chain cap)
  → click a row → the same Call detail drawer as the provider sees (shared component)
```

B4. Change the policy

```
Agent detail → Policy → Edit
  Settl rules (allowlist, max per call, daily budget) → Save → applied instantly
  On-chain rules (cap, expiry) → Save → "Settling on Stellar…" → ✓ (a new transaction)
```

B5. Approve a large spend (stretch)

```
Notification "Bookkeeper wants to pay ₱12 to GeoPH (over your ₱5 auto-approve)"
  → [Approve once] / [Deny] → the agent continues or gets a "declined" result
```

B6. Revoke an agent

```
Agent detail → [Revoke agent] → Confirm: "Stops all payments and returns ₱38 to your wallet."
  → 3-step progress:
      ① Stop agent (revoke_agent)          ✓
      ② Return ₱38 (sweep_on_revoke)       ✓  tx ↗
      ③ Remove identity (device revoke)    ✓
  → Status: Revoked · Budget ₱0 · history kept (read-only)
```

1.3 Agent (system flow, what it decides)

```
Agent task: "parse receipt 3"
  → call ResiboAI /parse-receipt
  → response?
     200 → use result (free endpoint)
     402 → read quote (price, mode, criteria)
        → Policy check (off-chain): provider allowed? price ≤ max per call? daily budget left?
             ✗ → DECLINED → tool error "over budget / not allowed" → agent skips or flags it for a human
             ✓ → mode?
                per-call → sign + submit authorize → wait ~5s
                           on-chain rejects (cap/expiry/balance) → BLOCKED
                           ✓ → retry with proof → result + receipt (PAID)
                outcome  → pre-sign authorize (not sent) → retry with signed auth
                           → provider runs → verifier checks
                               pass → submitted → result + receipt (PAID)
                               fail → discarded → result marked unverified (NOT CHARGED)
                                      → agent decides: retry with a better image / skip / flag for a human
  → record expense → next receipt
```

Every arrow emits an event that the agent log, the buyer activity view, the provider feed and the stage view all show.

1.4 Verifier (system flow)

```
Receive { quote, signed auth, provider result }
  → Match: signed args == quote (agent, device, merchant, amount, nonce)?  ✗ → REJECT (no checks)
  → Run checks for the criteria (schema, fields, confidence, totals, date)
  → pass  → submit signed authorize via Velo gas → PAID → record + feed event
  → fail  → discard signed auth → NOT CHARGED → record reasons + feed event
  → error → no charge → flag for review
```

1.5 Visitor or judge

```
Landing
  hero: "Let AI agents pay your API, only for results that pass."
  → [Watch live demo] → Stage view (read-only, auto-play loop)
  → [I sell an API] → Provider dashboard
  → [I run AI agents] → Buyer console
  → any tx link → stellar.expert (proof it's real)
```

1.6 Presenter (you, on stage, following spec §13)

```
Before going on stage: pnpm seed → open 3 tabs: Stage · Provider · Buyer
Stage view:
  [▶ Run receipt 1: clean]      → lane all ✓ → ResiboAI +₱2 → budget ₱48
  [▶ Run receipt 2: blurry]     → lane stops at Verify ✗ → "Not charged"
  [▶ Run receipt 3: Pro ₱15]    → lane stops at Pay ⛔ → "Blocked by on-chain policy"
→ Buyer tab: [Top up ₱50] → Velo checkout → ✓ balance updates
→ Provider tab: Earnings → [Claim] → ✓ received + tx
→ Close on Landing hero
Fallback: the stage view has [Replay recorded run] if testnet is down
```

1.7 Screens these flows need

Surface	Screens
Landing	Hero + 3 CTAs, "how it works" lane, 5-line snippet, footer with testnet contract IDs
Provider	Overview, Earnings, Endpoints (list + add/edit), Quality, Call detail drawer, empty states
Buyer	Agents list, New agent, Credentials dialog, Agent detail (budget, policy, activity), Top-up flow, Revoke dialog
Stage	Split view: agent thinking, earnings and budget, lane, tx ticker, run buttons, replay fallback
Shared	Top bar with role switcher + testnet badge + theme toggle, Lane, Status badge, Amount, Tx link, Budget meter, Check list, Settling indicator, Toasts

PART 3

Implementation Plan

- **Spec:** `docs/superpowers/specs/2026-11-03-settl-design.md`
- **Status:** FINAL v1.0
- **Platform:** responsive web app (Next.js); no native mobile app
- **Team:** solo
- **Window:** Thu Nov 5 (kickoff) to Sun Nov 8, 8:00 AM Manila (submission)
- **Rule:** no product code before kickoff. Day 0 is setup and reading only.

---

0. Assumptions (defaults for the spec's open questions)

Spec Q	Default used in this plan	Change cost if you disagree
Q1 Name	`settl`	Rename only
Q2 LLM	AI SDK with the model set in `AGENT_MODEL` env, so any provider you have a key for works	None
Q3 Wallets	Custodial testnet keypairs, encrypted at rest (`WALLET_ENC_KEY`)	+1 day for non-custodial
Q4 Concurrency	Payments sent one at a time per agent	Low
Q5 Hosting	Vercel (web, resibo) and Neon Postgres	Low

Other decisions:

- **Package manager:** pnpm workspaces
- **Tests:** Vitest
- **DB:** Prisma with Neon Postgres
- **Validation:** Zod
- **Amounts:** `bigint` in stroops (7 decimals) internally, decimal strings on the wire
- **Testnet-only tests** are tagged and run with `pnpm test:testnet`. The default `pnpm test` never touches the network.

---

1. Repository layout

```
settl/
├─ package.json                # workspaces, root scripts
├─ pnpm-workspace.yaml
├─ tsconfig.base.json
├─ vitest.workspace.ts
├─ .env.example                # every env var below, with placeholder values
├─ docs/
│  ├─ spikes.md                # findings from S1–S7 (Day 1)
│  └─ demo-script.md
├─ spikes/                     # throwaway scripts for S1–S7
│  ├─ s1-third-party-auth.ts
│  ├─ s2-max-amount.ts
│  ├─ s3-gas-soroban.ts
│  ├─ s4-escrow-token.ts
│  ├─ s5-fund-escrow-signer.ts
│  ├─ s6-nonce-rules.ts
│  └─ s7-checkout-settlement.ts
├─ packages/
│  ├─ core/                    # shared types, Zod schemas, encoding, config
│  │  └─ src/{index,protocol,schemas,amount,deviceHash,config,errors}.ts
│  ├─ stellar/                 # Noir contract client and Velo gas wrapper
│  │  └─ src/{index,noir,gas,keys,rpc}.ts
│  ├─ verifier/                # deterministic criteria checks
│  │  └─ src/{index,runChecks}.ts, src/criteria/resibo-receipt-v1.ts
│  ├─ sdk/                     # settl-sdk (provider middleware)
│  │  └─ src/{index,middleware,settlApi}.ts
│  └─ client/                  # settl-client (agent side)
│     └─ src/{index,settlFetch,policy,pay}.ts
└─ apps/
   ├─ web/                     # Next.js: dashboards, console, Settl API
   │  ├─ prisma/schema.prisma
   │  ├─ prisma/seed.ts
   │  ├─ app/(provider)/provider/...      # dashboard pages
   │  ├─ app/(buyer)/buyer/...            # console pages
   │  ├─ app/api/quotes/route.ts
   │  ├─ app/api/payments/verify/route.ts
   │  ├─ app/api/verify/route.ts
   │  ├─ app/api/velo/webhook/route.ts
   │  ├─ app/api/agents/route.ts
   │  ├─ app/api/agents/[id]/topup/route.ts
   │  ├─ app/api/agents/[id]/revoke/route.ts
   │  ├─ app/api/providers/[id]/claim/route.ts
   │  ├─ app/api/feed/route.ts            # SSE live feed
   │  └─ lib/{db,auth,wallets,services/*}.ts
   ├─ resibo/                  # demo provider (Express), uses settl-sdk
   │  └─ src/{server,parseReceipt,fixtures}.ts
   └─ agent/                   # demo bookkeeping agent
      └─ src/{run,tools,log}.ts
```

2. Environment variables (`.env.example`)

Var	Used by	Notes
`STELLAR_RPC_URL`	stellar	Soroban testnet RPC
`STELLAR_NETWORK_PASSPHRASE`	stellar	Testnet passphrase

Var	Used by	Notes
`NOIR_DEVICE_REGISTRY_ID` / `NOIR_AGENT_REGISTRY_ID` / `NOIR_PAYMENT_ESCROW_ID`	stellar	Latest IDs from the hackathon form (spec §8.2), not the Noir README
`VELO_API_KEY`	web, stellar	From velo-build.dev
`VELO_WEBHOOK_SECRET`	web	Webhook HMAC secret
`DATABASE_URL`	web	Neon
`WALLET_ENC_KEY`	web	32-byte key for custodial testnet secrets
`SETTL_API_URL` / `SETTL_PROVIDER_KEY`	resibo (sdk)	The provider authenticates to the Settl API
`AGENT_MODEL` + provider key	agent	Any AI SDK model string
`AGENT_SECRET` / `AGENT_DEVICE_HASH`	agent	Printed by the seed script

3. Frozen shared interfaces (write these first on Day 1, change only with a note in this file)

3.1 `packages/core/src/protocol.ts`

```
export type Mode = "per-call" | "outcome";
export type Amount = string; // decimal string, 7dp max, e.g. "2.0000000"

export interface Criteria {
  schema: string;                       // e.g. "resibo.receipt.v1"
  required?: string[];
  minConfidence?: number;
  lineItemsSumToTotal?: { tolerance: number };
}

export interface Quote {
  settl: "0.1";
  quoteId: string;
  mode: Mode;
  network: "stellar:testnet";
  asset: string;                        // escrow token contract id
  amount: Amount;
  displayPrice: { currency: "PHP"; amount: string };
  merchant: string;                     // G... provider wallet
  escrowContract: string;
  nonce: string;                        // u64 as string
  expiresAt: string;                    // ISO
  resource: string;                     // "POST /parse-receipt"
  description: string;
  criteria?: Criteria;                  // required when mode === "outcome"
  verifier?: string;
}

export type PaymentProof =
  | { quoteId: string; kind: "tx";   txHash: string;       agent: string; deviceHash: string }
  | { quoteId: string; kind: "auth"; authEntryXdr: string; agent: string; deviceHash: string };

export interface Receipt {
  quoteId: string;
  status: "paid" | "not_charged";
  txHash?: string;
  amount: Amount;
  verified: boolean;
  remainingBudget?: Amount;
  reasons?: CheckFailure[];
}

export interface CheckFailure { check: string; message: string }

export interface VerificationResult {
  result: "pass" | "fail" | "error";
  reasons: CheckFailure[];
}

export const HEADER_PAYMENT = "X-PAYMENT";
export const HEADER_RECEIPT = "X-PAYMENT-RECEIPT";
```

3.2 `packages/core` helpers

```
encodeHeader(obj: unknown): string                // base64(JSON)
decodeHeader<T>(raw: string, schema: ZodType<T>): T
toStroops(a: Amount): bigint
fromStroops(s: bigint): Amount
deviceHashForAgent(agentId: string): string       // hex sha256("settl:agent:" + agentId)
QuoteSchema, PaymentProofSchema, ReceiptSchema, CriteriaSchema   // Zod
```

3.3 `packages/stellar/src/noir.ts`

```
export interface TxResult { hash: string; status: "SUCCESS" | "FAILED"; error?: string }
export interface PolicyView { owner: string; agent: string; maxAmount: bigint; asset: string; expiresAt: bigint }

export interface NoirClient {
  // device_registry
  registerDevice(owner: Keypair, deviceHash: string, agent: string): Promise<TxResult>;
  revokeDevice(owner: Keypair, deviceHash: string): Promise<TxResult>;
  // agent_registry
  registerAgent(owner: Keypair, p: { deviceHash: string; agent: string; maxAmount: bigint; asset: string; expiresAt:
bigint }): Promise<TxResult>;
  revokeAgent(owner: Keypair, deviceHash: string): Promise<TxResult>;
  getPolicy(deviceHash: string): Promise<PolicyView | null>;
  // payment_escrow
  escrowToken(): Promise<string>;
  fundEscrow(owner: Keypair, deviceHash: string, amount: bigint): Promise<TxResult>;
  authorize(agent: Keypair, a: AuthorizeArgs): Promise<TxResult>;              // per-call
  preSignAuthorize(agent: Keypair, a: AuthorizeArgs): Promise<string>;         // outcome: auth entry XDR
  submitPreSigned(submitter: Keypair, authEntryXdr: string, a: AuthorizeArgs): Promise<TxResult>;
  claim(merchant: Keypair): Promise<TxResult>;
  sweepOnRevoke(owner: Keypair, deviceHash: string, agent: string): Promise<TxResult>;
  balanceOf(deviceHash: string): Promise<bigint>;
  pendingBalance(merchant: string): Promise<bigint>;
  // verification helper
  readAuthorizeTx(txHash: string): Promise<{ ok: boolean; args?: AuthorizeArgs }>;
}

export interface AuthorizeArgs {
  agent: string; deviceHash: string; merchant: string; asset: string; amount: bigint; nonce: bigint;
}
```

All transactions go through `gas.ts` (`sponsorAndSubmit` → `waitForResult`) unless Spike S3 says Soroban sponsorship doesn't work. The fallback is direct submission, with fees paid in testnet XLM.

3.4 Settl API (consumed by the SDK; Bearer `SETTL_PROVIDER_KEY`)

Method & path	Body	Returns
`POST /api/quotes`	`{ endpointId, agentHint?: { agent, deviceHash } }`	`Quote`
`POST /api/payments/verify`	`{ proof: PaymentProof }` (kind `tx`)	`{ ok: boolean; receipt?: Receipt; error?: string }`
`POST /api/verify`	`{ proof: PaymentProof /* kind auth */; result: unknown }`	`{ verification: VerificationResult; receipt: Receipt }`

3.5 SDK and client public API

```
// settl-sdk
export function settl(opts: { endpointId: string; apiUrl?: string; apiKey?: string }): RequestHandler; // Express

// settl-client
export async function settlFetch(url: string, init: RequestInit & {
  agent: Keypair; deviceHash: string;
  policy: { maxPrice: Amount; allowMerchants: string[] };
  onEvent?: (e: ClientEvent) => void;    // for the agent's reasoning log
}): Promise<{ response: Response; receipt?: Receipt }>;
```

---

4. Day 0: Nov 3–4 (no product code)

- [ ] Fill in the Google registration form and upload screenshots. **Deadline: Nov 4, 11:59 PM Manila.**
  - [ ] Star `Velo-Ecosystem-Collection/Velo` and `noirx402/noir`
  - [ ] Follow Velo, Noir, @highrable, @GigPayPH and @VertigralTech on X
- [ ] Sign up at velo-build.dev, register the project, and get the testnet API key and webhook secret.
- [ ] Read the hackathon rules: is pre-kickoff code allowed? Is a public repo or video required?
- [ ] Read the Noir contract source (`device_registry`, `agent_registry`, `payment_escrow`) and write down guesses for S1–S7.
- [ ] Read Velo's `docs/` and the SDK README. Check whether `gas.sponsor` covers `invokeHostFunction` (S3).
- [ ] Create a Neon project and get an LLM provider key.
- [ ] Collect 4 receipt images for fixtures: 2 clean, 1 blurry, 1 with "wrong totals".
- [ ] Install Node 22+, pnpm and the Stellar CLI.

---

5. Day 1: Thu Nov 5. Spikes and the per-call payment path

Day-1 goal: a real agent pays a real ResiboAI call on testnet in per-call mode.

T1: Scaffold the monorepo (30 min)

- **Files:** root `package.json`, `pnpm-workspace.yaml`, `tsconfig.base.json`, `vitest.workspace.ts`, `.env.example`, an empty `package.json` and `src/index.ts` in each package and app.
- **Steps:** set up pnpm workspaces, strict TS, Vitest, and ESLint with the TS preset.
- **Verify:** `pnpm -r typecheck && pnpm test` passes (an empty suite is OK).

T2: Spikes S1–S7 (2–3 h, a decision gate)

- **Files:** `spikes/s*.ts`, `docs/spikes.md`
- **Steps:** one script per spike against the testnet contracts. Record the outcome, any evidence (tx hashes) and the decision in `docs/spikes.md`.
- **Decisions that come out of it:**
  - **S1** (third-party submission): keep outcome mode as designed, or use the fallback where the client submits after a pass.

  - **S3** (Soroban gas sponsorship): Velo gas, or Friendbot-funded agents.
  - **S5** (who can fund escrow) and **S7** (where checkout settles): the top-up path.
  - **S6** (nonce rules): how nonces are allocated.
- **Verify:** every row in `docs/spikes.md` has an outcome and a decision. Update §3 of this file if an interface changes.

T3: `packages/core` (1 h, TDD)

- **Tests first** (`packages/core/test/*.test.ts`):
  - `toStroops("2") === 20000000n` . Round trips work. Rejects more than 7 decimals and negatives.
  - `encodeHeader` and `decodeHeader` round-trip a `PaymentProof` . Bad base64 or the wrong shape throws `SettlError("bad_header")` .
  - `QuoteSchema` rejects `mode: "outcome"` without `criteria` .
  - `deviceHashForAgent("a1")` is stable, 64 hex characters.
- **Implement:** `protocol.ts` , `schemas.ts` , `amount.ts` , `deviceHash.ts` , `errors.ts` , `config.ts` (contract IDs from env).
- **Verify:** `pnpm --filter core test`

T4: `packages/stellar` (2 h)

- **Tests:**
  - Unit (mocked RPC): `noir.ts` builds the right contract call and argument types for each method, including `i128` amounts and `BytesN<32>` device hashes. `gas.ts` maps Velo errors to `TxResult.error` .
  - Testnet (`*.testnet.test.ts`): a full cycle of register device, register agent, fund, authorize, claim, read balances, revoke agent, sweep, revoke device (in this order).
- **Implement:** `rpc.ts` (Soroban RPC client, simulate, assemble), `noir.ts` (the `NoirClient` from §3.3), `gas.ts` (Velo wrapper), `keys.ts` (Friendbot funding, keypair helpers).
- **Verify:** `pnpm --filter stellar test` and `pnpm --filter stellar test:testnet`

T5: Web app skeleton and DB (1 h)

- **Files:** `apps/web` (Next.js App Router), `prisma/schema.prisma` (tables from spec §11: `Org` , `Agent` , `Policy` , `Provider` , `Endpoint` , `Quote` , `Payment` , `Verification` , `Topup` , `Claim` , `WebhookEvent`), `lib/db.ts` , `lib/wallets.ts` (encrypt and decrypt custodial secrets).
- **Tests:** `lib/wallets.test.ts` : the encrypt/decrypt round trip works, and a wrong key fails.
- **Verify:** `pnpm --filter web prisma migrate dev` , `pnpm --filter web build`

T6: Quote and per-call verification services (1.5 h, TDD)

- **Files:** `apps/web/lib/services/quotes.ts` , `lib/services/payments.ts` , `app/api/quotes/route.ts` , `app/api/payments/verify/route.ts` , `lib/auth.ts` (provider key check)
- **Tests first** (NoirClient mocked):
  - The quote has the endpoint's price, the provider's wallet as merchant, a unique nonce (per the S6 decision) and an expiry 5 min out.
  - Verify accepts when the tx succeeded **and** its args match the quote (agent, device, merchant, asset, amount, nonce). The quote becomes `paid` and a `Payment` row is written.

- It rejects: an expired quote, amount mismatch, merchant mismatch, a reused quote (`paid` already), a failed tx, and a missing or bad provider key (401).

- **Verify:** `pnpm --filter web test`

T7: `packages/sdk` per-call middleware (1 h, TDD)

- **Tests first** (Settl API mocked with `msw`):
  - No `X-PAYMENT` header: 402 with a body that validates against `QuoteSchema`.
  - Valid tx proof: calls `next()` and sets `X-PAYMENT-RECEIPT`.
  - Verify returns `ok: false`: 402 again, with `error`.
  - Malformed header: 400.
- **Implement:** `middleware.ts`, `settlApi.ts`
- **Verify:** `pnpm --filter sdk test`

T8: `packages/client` per-call `settlFetch` (1 h, TDD)

- **Tests first** (fetch and NoirClient mocked):
  - First call returns 200: returned as is, no payment.
  - 402 within policy: `authorize` is called once, the retry carries `X-PAYMENT`, and the receipt is parsed.
  - 402 where the price is above `maxPrice`, or the merchant isn't allowed: throws `PolicyError`, and no payment is made.
  - The `onEvent` sequence is `quote → policy_ok → paying → paid → retry`.
  - Payments are sent one at a time per agent (Q4): two parallel calls don't overlap `authorize`.
- **Implement:** `policy.ts`, `pay.ts`, `settlFetch.ts`
- **Verify:** `pnpm --filter client test`

T9: ResiboAI and the end-to-end per-call path (1 h)

- **Files:** `apps/resibo/src/server.ts` (Express, `POST /parse-receipt` behind `settl({ endpointId })`), `parseReceipt.ts` (picks a fixture by image hash, defaulting to "clean"), `fixtures.ts` (clean / blurry / wrong-totals JSON outputs).
- **Files:** `apps/web/prisma/seed.ts` (minimal for now: a provider org with a Friendbot-funded wallet, the ResiboAI endpoint at ₱2 per-call, and a buyer org with one agent registered, a policy and funding). It prints `AGENT_SECRET` and `AGENT_DEVICE_HASH`.
- **Script:** `apps/agent/src/smoke.ts` calls ResiboAI through `settlFetch` with a clean receipt.
- **Verify (gate):** the smoke script prints the result plus a receipt with a tx hash, and `pendingBalance(provider)` went up by 2. **If this fails, Day 2 starts here.**

---

6. Day 2: Fri Nov 6. Buyer console, top-ups and the demo agent

Day-2 goal: a buyer can create an agent, set its policy and fund it through Velo checkout from the UI, and an LLM agent spends it.

T10: Agent lifecycle service (1.5 h, TDD)

- **Files:** `lib/services/agents.ts`, `app/api/agents/route.ts`, `app/api/agents/[id]/revoke/route.ts`

- **Tests first** (NoirClient mocked):
  - Create: generates a keypair, computes `device_hash`, then calls `registerDevice` and then `registerAgent` with the policy. It stores the agent and policy. If `registerAgent` fails, the agent is marked `error`, not `active`.
  - Revoke: calls in the order `revokeAgent → sweepOnRevoke → revokeDevice` (assert the call order). It refuses `revokeDevice` if `balanceOf > 0` after the sweep.
  - The off-chain policy fields (allowlist, max price, daily budget) are saved and returned.
- **Verify:** `pnpm --filter web test`

T11: Velo top-up (2 h, TDD)

- **Files:** `lib/services/topups.ts`, `app/api/agents/[id]/topup/route.ts`, `app/api/velo/webhook/route.ts`
- **Tests first:**
  - Top-up creates a checkout session (Velo SDK mocked) and a `Topup` row with status `pending`, and returns `checkoutUrl`.
  - Webhook: a bad signature gets 401. A valid one confirms via `paymentIntents.retrieve`, then calls `fundEscrow` and marks the top-up `funded` with the tx hash.
  - Idempotency: the same event ID twice funds exactly once (`WebhookEvent` unique on event ID).
  - Raw body: the signature is checked against the raw request body, not re-serialized JSON.
- **Implement** per the S5 and S7 decisions. If checkout settles to a treasury, the worker forwards to the org wallet before `fundEscrow`.
- **Verify:** `pnpm --filter web test`. Then a manual end-to-end with the Velo testnet checkout: the balance goes up in `balanceOf`.

T12: Buyer console UI (2 h)

- **Pages:** `app/(buyer)/buyer/agents/page.tsx` (list: balance, policy, status), `agents/new/page.tsx` (form: name, cap, expiry, allowlist, max price), `agents/[id]/page.tsx` (detail: Top up button opens Velo checkout, activity table with tx links, Revoke with a confirmation).
- **Craft bar:** clear empty states, loading and error states for every chain call, amounts shown in PHP with the token amount underneath, and tx links to stellar.expert (testnet).
- **Verify:** click through in the browser: create an agent, top up, see the balance, revoke, see the balance return.

T13: Demo agent (2 h)

- **Files:** `apps/agent/src/run.ts` (an AI SDK `generateText` or agent loop with tools), `tools.ts` (the `parse_receipt` tool calls `settlFetch`, and the `record_expense` tool appends to a local ledger), `log.ts` (a pretty terminal log of `onEvent` plus the model's reasoning).
- **Behaviour:** given a folder of receipt images, the agent parses each one and records the expense. Its log shows quote, policy decision, payment and receipt.
- **Tests:** `tools.test.ts`, with `settlFetch` mocked: a policy error comes back as a tool error, so the model sees "declined: over budget" and the agent doesn't crash.
- **Verify:** `pnpm --filter agent start ./fixtures/receipts` processes 2 clean receipts on testnet, and the log reads like the demo script.

T14: Seed for the demo (30 min)

- Extend `seed.ts` so one command builds the whole demo state: a ResiboAI provider with 2 endpoints (`/parse-receipt` outcome ₱2, and `/parse-receipt-pro` per-call priced **above** the agent's cap for the guardrail demo), and Juan's Accounting Firm with one agent (cap and ₱50 funded).
- **Verify:** `pnpm seed` on a fresh DB, then `pnpm --filter agent start` works with no manual steps.

---

7. Day 3: Sat Nov 7. Outcome mode, verifier and provider dashboard

Day-3 goal: the full demo script (spec §13) runs end to end, and a backup video is recorded.

T15: `packages/verifier` (1.5 h, TDD)

- **Tests first** (`criteria/resibo-receipt-v1.test.ts` , fixtures from `apps/resibo/src/fixtures.ts` via a shared test fixture):
  - Clean fixture: `pass` , with no reasons.
  - Blurry fixture: `fail` , with reasons including `confidence` .
  - Wrong-totals fixture: `fail` , with reason `lineItemsSumToTotal` .
  - Missing `date` : `fail` with `required` . A future date: `fail` with `dateSanity` .
  - Non-object or garbage input: `fail` with `schema` . Never throws.
  - `runChecks` with an unknown `criteria.schema` returns `error` .
- **Implement:** a Zod receipt schema plus the check functions. `runChecks(criteria, result): VerificationResult` .
- **Verify:** `pnpm --filter verifier test`

T16: Outcome mode end to end (2.5 h, TDD)

- **Web** `lib/services/verify.ts` + `app/api/verify/route.ts` , tests first:
  - Pass: `submitPreSigned` is called with the quote's args, the quote becomes `verified` , and the receipt status is `paid` .
  - Fail: `submitPreSigned` is **not** called, the `Verification` row stores the reasons, and the receipt status is `not_charged` .
  - Proof args that don't match the quote: rejected before any checks run.
  - Verifier exception: `error` , no charge.
- **SDK** `middleware.ts` , tests first: in outcome mode it runs the handler, captures the JSON body, posts it to `/api/verify` , then responds with the result and the receipt header (200 on pass; 200 with `status: not_charged` on fail, per spec §6.2.6).
- **Client** `pay.ts` , tests first: in outcome mode it calls `preSignAuthorize` and sends `kind: "auth"` . It never calls `authorize` directly.
- **If S1 failed (fallback):** the verify response includes `submit: true` , and the client submits `authorize` itself. Test both paths behind a single flag.
- **Verify:** the agent run against the clean receipt shows `paid` , and against the blurry one shows `not_charged` , with `pendingBalance` changing only on the clean one.

T17: Provider dashboard (3 h)

- **Pages:** `app/(provider)/provider/page.tsx` (overview: revenue today and all time, calls, pass rate, live feed), `earnings/page.tsx` (pending from `pendingBalance` vs claimed, **Claim** button, claim history), `endpoints/page.tsx` (price, mode, criteria, SDK snippet to copy), `quality/page.tsx` (fail counts per check).
- **API:** `app/api/providers/[id]/claim/route.ts` (calls `claim` with the custodial provider wallet and writes a `Claim` row), `app/api/feed/route.ts` (SSE stream of new Payment and Verification rows on the Node runtime).
- **Tests:** a claim service test: a successful claim writes a row, and a failed tx returns an error with no row. Feed: a smoke test that it emits an event after a payment insert.
- **Craft bar:** the live feed animates new rows. Pass and fail are badged with both colour and text. Big readable numbers for the stage projector. Every amount links to its tx.
- **Verify:** with the agent running, the dashboard updates live, and Claim moves pending to claimed with a tx link.

T18: Guardrail path (45 min)

- Make sure that when the agent calls `/parse-receipt-pro`, either the client policy rejects it, or (with the client check turned off for the demo) the on-chain `check_payment` rejects `authorize`. The failure should show as a clean "blocked by on-chain policy" event in the agent log and the buyer activity view.
- **Test:** client test that an on-chain rejection maps to `PolicyError("onchain_policy")`.

T19: Deploy and record the backup video (1.5 h)

- Deploy `apps/web` and `apps/resibo` to Vercel. Set the env vars and run `prisma migrate deploy`, then the seed.
- Point the Velo webhook at the deployed `/api/velo/webhook`.
- Run the full demo (`docs/demo-script.md`) on the deployed version, then screen-record it with a voice-over.
- **Verify:** the video file exists, and every step of spec §13 is visible in it.

---

8. Day 4: Sun Nov 8, before 8:00 AM. Verify and submit

T20: Full verification pass (1 h)

- `pnpm -r typecheck`
- `pnpm -r lint`
- `pnpm test` (all unit tests)
- `pnpm test:testnet` (the stellar cycle plus one per-call and one outcome payment)
- `pnpm -r build`
- A demo-script dry run on the deployed URLs
- Record the results in `docs/verification.md`. Fix or explicitly list any failures. Never submit with silent failures.

T21: Submission package (1 h)

- `README.md`: the problem, how it works (the diagram from spec §5), how to run locally, testnet contract IDs, Velo features used, limitations (spec §14), and the roadmap.
- Links: deployed app, ResiboAI, the demo video, and example tx hashes on stellar.expert.
- Submit by **8:00 AM**, and leave at least 30 minutes of buffer.

T22: Pitch prep (between 8 AM and 1 PM)

- A 3-minute live demo and a 1-minute backup video clip.
- One slide each: the problem, how it works, the demo, the business model, the roadmap.
- Pre-load the demo state, with the seed run fresh, the agent funded and the dashboards open in tabs.

T23: Stretch. Spend approval page (about half a day, only if the must-haves are done)

- **Files:** `apps/web/app/approve/[id]/page.tsx` (phone-sized, opened from a link), `lib/services/approvals.ts`, `app/api/approvals/[id]/route.ts`, plus an `Approval` table (id, quoteId, agentId, amount, status `pending` / `approved` / `denied`, decidedAt).
- **Behaviour:** when a quote is over the agent's auto-approve threshold, `settlFetch` creates an approval and waits (polling, 60s timeout). Approve resumes the payment. Deny or timeout returns a `PolicyError("declined")`.
- **Tests first:** an approval is created above the threshold and not below it. Approve resumes exactly one payment. Deny and timeout make no payment. A decided approval can't be changed.
- **Demo:** on stage, the agent hits a ₱12 quote, your phone opens the approval page, you tap Approve, and the payment lands on the projector.
- **Verify:** `pnpm --filter web test`, then a manual run on a real phone.

---

9. Cut line (when behind schedule)

Cut in this order: daily budget → pricing editor in endpoints → quality page → buyer console polish → SSE (fall back to a 3s poll).

Never cut: T9 (per-call end to end), T15–T16 (verifier and outcome mode), Claim (T17), and the backup video (T19).

Checkpoint	If not met	Action
Thu 11 PM: T9 gate	No real paid call yet	Skip Velo gas (use Friendbot fees), and start Friday on T9
Fri 11 PM: T11 + T13	Top-up or agent not working	Seed funds the escrow directly and the top-up is shown as a click-through, so the agent takes priority
Sat 6 PM: T16	Outcome mode not working	Use the S1 fallback (the client submits after a pass)
Sat 11 PM: T19	No video	Record locally, before polishing anything

---

10. Task → spec coverage

Spec section	Tasks
§6.1 Per-call	T6, T7, T8, T9
§6.2 Outcome	T15, T16

Spec section	Tasks
§7 Wire protocol	T3, T7, T8
§8 Noir/Velo sequences	T4, T10, T11, T17
§9 Verifier	T15
§10 Policy	T8, T10, T18
§11 Data model	T5
§12 Screens	T12, T17
§13 Demo	T13, T14, T18, T19, T22
§15 Spikes	T2
§17 Timeline	§4–§8 of this plan